

CosineEmbeddingLoss#

<https://pytorch.org/docs/stable/generated/torch.nn.CosineEmbeddingLoss.html#>

Description:

Creates a criterion that measures the loss given input tensors x_1 ($x_1 \times 1$), x_2 ($x_2 \times 2$) and a Tensor label y with values 1 or -1. Use $(y=1)$ to maximize the cosine similarity of two inputs, and $(y=-1)$ otherwise. This is typically used for learning nonlinear embeddings or semi-supervised learning. The loss function for each sample is: margin (float, optional) – Should be a number from -1 to 1, 0.5 is suggested. If margin is missing, the default value is 0. size_average (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field size_average is set to False, the losses are instead summed for each minibatch. Ignored when reduce is False. Default: True reduce (bool, optional) – Deprecated (see reduction). By default, the losses are averaged or summed over observations for each minibatch depending on size_average. When reduce is False, returns a loss per batch element instead and ignores size_average. Default: True

Function Signature

```
class torch.nn.CosineEmbeddingLoss(margin=0.0,  
size_average=None, reduce=None, reduction='mean')[source]#
```

Parameters

Shape:

```
forward(input1, input2, target)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> loss = nn.CosineEmbeddingLoss()  
>>> input1 = torch.randn(3, 5, requires_grad=True)  
>>> input2 = torch.randn(3, 5, requires_grad=True)  
>>> target = torch.ones(3)  
>>> output = loss(input1, input2, target)  
>>> output.backward()
```

Detailed Documentation

Documentation

Creates a criterion that measures the loss given input tensors x_1 ($x_1 \times 1$), x_2 ($x_2 \times 2$) and a Tensor label y with values 1 or -1. Use $(y=1)$ to maximize the cosine similarity of two inputs, and $(y=-1)$ otherwise. This is typically used for learning nonlinear embeddings or semi-supervised learning.

The loss function for each sample is:

margin (float, optional) – Should be a number from -1 to 1 , 0.5 is suggested. If margin is missing, the default value is 0 .

size_average (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field `size_average` is set to `False`, the losses are instead summed for each minibatch. Ignored when `reduce` is `False`. Default: `True`

reduce (bool, optional) – Deprecated (see reduction). By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`. When `reduce` is `False`, returns a loss per batch element instead and ignores `size_average`. Default: `True`

reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: `size_average` and `reduce` are in the process of being deprecated, and in the meantime, specifying either of those two args will override `reduction`. Default: 'mean'

Input1: (N,D) , (N, D) , (N,D) or $(D)(D)(D)$, where N is the batch size and D is the embedding dimension.

Input2: (N,D) , (N, D) , (N,D) or $(D)(D)(D)$, same shape as Input1.

Target: (N)(N)(N) or {}{}.

Output: If reduction is 'none', then (N)(N)(N), otherwise scalar.

Runs the forward pass.